Microsoft Defender for Endpoint — Network Telemetry Visibility in Web Proxy / SWG Architectures (iboss, Zscaler, Loopback)

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 6

Boundary . . . 6

Effort . . . 6

1. Solution Overview — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures 6

What this solution is . . . 6

How the technology behaves (why destination telemetry is lost) . . . 7

Where this matters in the Defender stack . . . 7

The X-Forwarded-For (XFF) clarification (in brief) . . . 8

The standard deployment model . . . 8

Scope . . . 9

What is out of scope (cross-reference, don't duplicate) . . . 9

Connections and authentication used . . . 9

Limitations (be honest with buyers) . . . 10

2. Requirements — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures 10

R1. Defender for Endpoint P2 with Advanced Hunting available . . . 10

R2. Onboarded devices behind the SWG, with warmed-up telemetry . . . 11

R3. Security Reader (read-only) access to advanced hunting . . . 11

R4. Identification of the SWG agent and its steering mode . . . 11

R5. The loopback / proxy address set for destination classification . . . 11

R6. Microsoft Graph PowerShell modules (Microsoft.Graph) . . . 12

R7. A device sample definition (the assessment ring) . . . 12

R8. A Microsoft support entitlement for the advisory case . . . 12

R9. (Optional) Log Analytics / SIEM export for longer-window analysis . . . 12

Global Requirements — Defender (category-level) 13

G1. Microsoft 365 / Entra tenant . . . 13

G2. Defender licensing . . . 13

G3. Administrative roles (least privilege) . . . 13

G4. Onboarding reach to endpoints / identity / workloads . . . 13

G5. Network access to Defender service endpoints . . . 13

G6. A pilot ring . . . 14

G7. Log Analytics workspace (where required) . . . 14
G8. Response governance . . . 14
3. Implementation — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures 14
Stage 1 — Assess (environment + sample) . . . 15
Stage 2 — Configure (measurement parameters) . . . 15
Stage 3 — Pilot (measure the sample, read-only) . . . 16
Stage 4 — Validate (findings, matrix, XFF) . . . 16
Stage 5 — Expand (estate assessment + remediation plan + advisory case) . . . 18
5. Testing Guide — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures 20
Part A — Assessment readiness . . . 20
Part B — Measurement validity . . . 20
Part C — Deliverable integrity (honest framing) . . . 21
6. Troubleshooting — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures 22
No DeviceNetworkEvents data (can't measure obfuscation at all) . . . 22
Obscured share comes back 0% but you expected obfuscation . . . 22
Obscured share looks implausibly high (≈100% on everything) . . . 22
RemoteUrl is missing but RemoteIP looks like a real remote . . . 23
Completeness ratio is low but obfuscation is low too . . . 23
Feature enumeration returns nothing (indicators / detections empty) . . . 23
The matrix or report asserts a Microsoft answer . . . 23
The remediation reads as an applied change . . . 23
SWG steering mode can't be confirmed . . . 23
Sensor can reach the service, but you're still worried about connectivity . . . 24
Advisory case gets bounced to a higher/lower severity . . . 24
7. Best Practices — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures 24
Keep it read-only and keep it that way . . . 24
Make the measurement defensible . . . 24
Never invent Microsoft's answers . . . 25
Defer the decisions that aren't technical . . . 25
Operate the loop . . . 25
Where to go next (adjacent catalog areas) . . . 25
8. Deliverables Summary — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures 26
Documentation (source guides) . . . 26

Scripts (read-only assessment + reporting) 26
Named objects produced (documents, not changes) 27
How it is delivered 27
Appendix A — PowerShell Scripts 28

Executive Summary — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures

White-label · Defender

When a secure web gateway (SWG) — iboss, Zscaler, or a generic loopback/PAC agent — steers endpoint traffic through a **local listener** before it egresses, the Microsoft Defender for Endpoint (MDE) sensor can record the **proxy or loopback address as the destination** instead of the true remote server. The application opens its connection to `127.0.0.1` (or the agent's local proxy IP); the SWG then makes the real outbound connection on the endpoint's behalf. From the sensor's point of view, "where did this device connect?" answers *the loopback listener* — so `DeviceNetworkEvents` fills with obscured destinations, and the network-dependent Defender signals that read those destinations (SmartScreen, Network Protection, threat-intelligence indicator matching, and any custom detection or hunt that pivots on `RemoteIP` / `RemoteUrl`) lose the very field they depend on.

This is **not a connectivity failure** — the device is protected on the wire and the sensor still reports to the Defender cloud. It is a **telemetry-visibility** problem: the endpoint's record of *what it talked to* is degraded, quietly, with no alert to say so. Teams usually discover it only when a hunt returns loopback IPs, an indicator that should have blocked didn't, or an investigation can't reconstruct a device's true network path.

This solution measures **exactly which telemetry your tenant is losing to the proxy**, documents **which features are affected and how**, clarifies the frequently-misunderstood **X-Forwarded-For** question, gives you a **supported-topology remediation** to take to your network and security owners, and hands you a **Microsoft advisory support-ticket template** so the authoritative answers — which tables, which XFF behavior, which supported coexistence — come from Microsoft, not from guesswork.

What you get

- A plain-language **Solution Overview** of how loopback/forward-proxy steering obscures endpoint destination telemetry, and where in the Defender stack that matters.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (the category-level Defender prerequisites live in the separate global-requirements file).
- An **Implementation Guide** of exact, ordered, **read-only** steps: scope a device sample → run the destination-obfuscation and completeness measurement → build the feature-impact matrix → produce the XFF clarification → draft the supported-topology remediation → open the Microsoft advisory case → expand the assessment across the estate.
- **Five scripts** — four PowerShell (Microsoft Graph Security advanced-hunting) and one KQL query pack — that **assess and report only**. They sample devices, measure `DeviceNetworkEvents` completeness and destination obfuscation, enumerate which network Advanced Hunting tables are populated, map network-dependent features/detections/indicators to the proxy behavior, and compile the assessment. **They change nothing** in the proxy or the tenant.
- **Testing** and **Troubleshooting** guides, a **feature-impact matrix**, an **XFF clarification** section, and a ready-to-send **Microsoft advisory ticket template**.

What this delivers

Clear, tenant-specific evidence of what network telemetry the proxy is costing you — measured, not assumed — plus a documented feature-impact matrix, an honest XFF clarification, a supported-topology remediation to hand to your change owners, and a Microsoft advisory case that confirms the authoritative behavior. You end with a decision-ready package: *this is what we're losing, this is why, this is the supported fix, and this is Microsoft's confirmation.*

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — the same repeatable pattern the whole catalog uses. Here it is applied to an **assessment**: assess the environment and scope a device sample, configure the measurement queries and thresholds, pilot the read-only measurement on the sample ring, validate the findings and the feature-impact matrix against explicit gates, then expand the assessment across the estate and open the Microsoft advisory case. Every step is **read-only**; the remediation is authored as a plan for the customer's change process, never applied by this package.

Boundary

This solution is about **telemetry visibility**, not connectivity, and not SWG deployment. It **cross-references rather than duplicates**: the **"Endpoint Network Connectivity Troubleshooting Toolkit"** (`NETDX` — DNS/proxy/VPN reachability, i.e. *can the sensor reach the service*, a different problem); the **Netskope SWG line** (`NSSWG`, `NSCASB`, and the SASE/SSE blueprint — *deploying and operating* a gateway); and the comprehensive **"Microsoft Defender for Endpoint"** (`MDEFULL`) and **"Microsoft Defender XDR — Unified Onboarding"** (`DEFXDR`) solutions (the full stand-up and cross-pillar onboarding). Use those where their scope fits; this solution is the focused **proxy-blind-telemetry assessment**. Microsoft's authoritative answers — which tables/fields are affected, whether XFF restores anything for endpoint telemetry, and which coexistence topology is supported — are **flagged "confirm with Microsoft / verify against current guidance" throughout and never asserted**. The decision to change SWG steering, accept a telemetry tradeoff, or re-architect the topology belongs to the customer's **network, security, and architecture** owners; this guide supplies the evidence and the technical options, not the decision.

Effort

A competent administrator can scope the device sample and run the read-only measurement in a focused session, compile the feature-impact matrix and assessment report, and draft the Microsoft advisory case the same day. The supported-topology remediation is authored here as a plan; applying it runs through the customer's normal network/security change process on its own timeline. The assessment is **reusable** — re-run it after any SWG mode change, connector upgrade, or Microsoft confirmation to re-measure the delta.

1. Solution Overview — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures

What this solution is

This is a **read-only telemetry-impact assessment** for Microsoft Defender for Endpoint (MDE) running alongside a **secure web gateway (SWG) that steers endpoint traffic through a local listener before egress** — the "loopback-forward" or "system-proxy" pattern used by agents such as iboss, Zscaler Client Connector, and generic PAC/loopback proxies. It measures how much of your endpoint **destination telemetry** the proxy is obscuring, documents which Defender features that affects, clarifies the X-Forwarded-For (XFF) question, drafts a supported-topology remediation for your change owners, and gives you a Microsoft advisory support-ticket template to get the authoritative answers.

It does **not** change the proxy, the SWG agent, or the tenant. Every script and step here **samples, measures, and reports**.

How the technology behaves (why destination telemetry is lost)

The MDE sensor records network activity **as the endpoint's own network stack sees it** — into `DeviceNetworkEvents` (and related fields on `DeviceEvents`), with fields like `RemoteIP`, `RemoteUrl`, `RemotePort`, and `RemoteIPType`. Those fields answer "what did this process connect to?"

When an SWG agent runs a **local loopback listener** and steers traffic to it (via a PAC file that points at `127.0.0.1:<port>`, a system-proxy setting, or a route/loopback tunnel the agent installs), the sequence becomes:

1. The application (browser, updater, tool) opens its connection to the **local listener** — e.g. `127.0.0.1`, `::1`, or the agent's local proxy IP — **not** the true remote server.

2. The SWG agent inspects/filters the request and then makes the **real** outbound connection to the true remote, off-box or via its own tunnel.

3. The MDE sensor, watching the endpoint's stack, records the connection the *application* made — so `RemoteIP` is the **loopback/proxy address** and `RemoteUrl` is missing or reflects the local listener, **not** the true remote destination.

The result: the sensor faithfully records a connection that is **technically accurate but analytically useless** for destination-based analysis — every app looks like it "talked to localhost." The true remote destination lives in the **SWG's** logs, not the endpoint's. This is why the problem is invisible: nothing is broken, the device is protected, the sensor reports — but the *destination dimension* of endpoint network telemetry is flattened to the proxy.

Exactly how, and how much, depends on the SWG's mode (route-based tunnel vs system-proxy/loopback, whether specific destinations are excluded from steering, whether TLS is decrypted locally). Vendor console internals and the precise per-mode behavior are **vendor-specific and version-specific — confirm against current vendor and Microsoft guidance; this solution measures the observed effect in your tenant rather than asserting the cause.**

Where this matters in the Defender stack

Any signal that reads the endpoint's record of the **true remote destination** is exposed:

- `DeviceNetworkEvents` **completeness & fidelity** — hunts and investigations that pivot on `RemoteIP` / `RemoteUrl` / `RemotePort` see loopback/proxy values instead of real destinations; network path reconstruction during IR is degraded.

• **SmartScreen** — reputation checks on URLs/downloads. Where the endpoint stack only ever sees the local listener, the destination-visibility SmartScreen and Network Protection rely on can be affected — *confirm current behavior with Microsoft.*

• **Network Protection** — blocks connections to malicious IPs/domains/URLs, including custom indicators and web content filtering. If the endpoint's outbound connection is to the loopback listener, whether Network Protection evaluates the **true** remote or the loopback destination is topology-dependent — *confirm with Microsoft.*

• **Threat-intelligence / custom indicator matching** — endpoint matching of IP / URL / domain indicators depends on the sensor seeing the true remote; obscured destinations can prevent endpoint-layer matches — *confirm with Microsoft.*

• `DeviceNetworkEvents`**-dependent detections** — built-in analytics and any custom detection rule keyed on remote destination inherit the same blind spot.

This solution **measures the first** (completeness/obfuscation is directly observable in your data) and **documents the rest as a feature-impact matrix flagged for Microsoft confirmation** — it does not assert Microsoft's definitive answer on SmartScreen / Network Protection / TI behavior under this topology.

The X-Forwarded-For (XFF) clarification (in brief)

`X-Forwarded-For` is a **server-side HTTP request header**: a proxy adds it so the **destination server** learns the original client's IP. It flows *forward* (proxy → server) and helps **server-side** attribution — the destination's logs, a WAF, the SWG's own records. It is **not** a mechanism that feeds the true remote destination back into the **endpoint's** MDE telemetry. Section 03 and the dedicated XFF clarification reason through what XFF does and does not restore for endpoint-originated telemetry — **explicitly framed "confirm current behavior with Microsoft," not asserted as Microsoft's definitive position.**

The standard deployment model

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand.*** *Assess the environment and scope the device sample; configure the measurement queries and thresholds; pilot the read-only measurement on the sample ring; validate the completeness/obfuscation findings and the feature-impact matrix against explicit gates; then expand the assessment across the estate and open the Microsoft advisory case. Every step is read-only.*

Applied here:

1. **Assess.** Confirm MDE plan/tables, identify the SWG agent and its steering mode (from your own config, not asserted), and scope a representative device sample.

2. **Configure.** Set the sample device group, the measurement window, and the loopback/proxy address set used to classify destinations.

3. **Pilot.** Run the read-only measurement on the sample ring — completeness of `DeviceNetworkEvents`, share of connections whose `RemoteIP` is loopback/proxy vs true remote, and which network tables are populated.

4. **Validate.** Confirm the findings against pass/fail gates, build the feature-impact matrix, and finalize the XFF clarification.

5. **Expand.** Roll the assessment across the estate, draft the supported-topology remediation for the change owners, and open the Microsoft advisory case to confirm the authoritative behavior.

Scope

• **Read-only measurement** of `DeviceNetworkEvents` completeness and **destination obfuscation** (loopback/proxy vs true remote) across a device sample, and of which network Advanced Hunting tables are populated.

• A **feature-impact matrix** mapping SmartScreen, Network Protection, threat-intelligence indicator matching, and `DeviceNetworkEvents`-dependent detections to the proxy behavior — flagged for Microsoft confirmation.

• An **XFF clarification** — what XFF is, why it is server-side, and what it does/doesn't restore for endpoint telemetry (confirm-with-Microsoft framing).

• A **supported-topology remediation** — proxy mode, exclusions, and connector/deployment options for a supported MDE + SWG coexistence, authored as a plan for the customer's change process and flagged "verify against current Microsoft guidance."

• A **Microsoft advisory support-ticket template** (Severity C) with the exact validate-telemetry / document-feature-impact / XFF / architecture-review / advanced-hunting-schema asks.

What is out of scope (cross-reference, don't duplicate)

• **Sensor-to-service connectivity** (can the MDE sensor reach the Defender cloud through the proxy; DNS/proxy/VPN reachability) — the **"Endpoint Network Connectivity Troubleshooting Toolkit"** (`NETDX`). *That is connectivity; this is telemetry visibility.*

• **Deploying, steering, and operating the SWG itself** — the **Netskope** line (`NSSWG` SWG deployment, `NSCASB` CASB, SSL/TLS inspection, traffic steering) and the SASE/SSE blueprint. This solution treats the SWG as a given and measures its telemetry effect.

• **The full MDE stand-up, tuning, and enablement** — the comprehensive **"Microsoft Defender for Endpoint"** solution (`MDEFULL`).

• **Cross-pillar XDR onboarding** (Identity, Office 365, Cloud Apps, Cloud) — **"Microsoft Defender XDR — Unified Onboarding"** (`DEFXDR`).

• **Changing SWG steering, applying the remediation, or accepting a telemetry tradeoff** — those are **decisions for the customer's network, security, and architecture owners**; this guide supplies the evidence and the options only.

• **Microsoft's authoritative determinations** — affected tables/fields, XFF behavior for endpoint telemetry, and the supported coexistence topology — are obtained via the advisory case, **not asserted here.**

Connections and authentication used

• **Advanced hunting via Microsoft Graph Security** (`Microsoft.Graph` PowerShell modules, `POST /security/runHuntingQuery`) for all read-only measurement over `DeviceNetworkEvents`, `DeviceEvents`, `DeviceInfo`, `DeviceProcessEvents`, `AlertInfo`, `AlertEvidence`. Least-privilege scope: `ThreatHunting.Read.All` (plus `SecurityEvents.Read.All` and

`Organization.Read.All` for context). No write scopes are used.

• The **unified Defender portal** (`security.microsoft.com` → **Hunting** → **Advanced hunting**, **Settings** → **Endpoints** → **Indicators / Advanced features**, **Assets** → **Devices**, **Custom detection rules**) for the manual enumeration steps (indicators, Network Protection state, custom detections) that are read-only checks.

• **Your SWG console** (iboss / Zscaler / other) is referenced **read-only** for identifying the steering mode and loopback/proxy address set — *its internals are vendor-specific; confirm against current vendor documentation.* This package never changes it.

• Least-privilege roles: **Security Reader** is sufficient for the read-only advanced-hunting and portal checks; no configuration role is required to run the assessment. Government-cloud tenants pass the matching Graph environment — *verify endpoints for your cloud.*

Limitations (be honest with buyers)

• Advanced hunting and full `DeviceNetworkEvents` require **Defender for Endpoint P2** — *verify current plan-to-feature mapping and availability in your tenant.*

• This assessment **measures the observed telemetry effect**; it does **not** assert *why* a given SWG mode produces it (vendor-specific) or Microsoft's definitive feature behavior (confirm via the advisory case).

• Advanced-hunting tables retain a **limited window** (commonly ~30 days — *verify current retention*); the measurement reflects that window, not all history.

• Destination classification relies on the **loopback/proxy address set you provide** — an incomplete set understates obfuscation; the scripts make the set explicit and reviewable.

• The **remediation is a plan**, not an applied change — SWG mode changes trade filtering coverage against endpoint visibility and are a **customer decision** executed through their own change process.

• Restoring endpoint destination visibility does **not** replace the SWG's own logs, the sensor's connectivity, patching, or the SOC's investigation — it restores one dimension of endpoint network telemetry.

2. Requirements — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures

The category-level prerequisites (tenant, Defender licensing, admin roles, onboarding reach, service-endpoint network access, a pilot ring, Log Analytics workspace where used, and response governance) are in the separate **Global Requirements — Defender** file published with this solution. **Do not repeat them; confirm them there first.** This section lists only the prerequisites **specific to this read-only telemetry assessment**. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. Defender for Endpoint P2 with Advanced Hunting available

Description: MDE **P2** so `DeviceNetworkEvents` and the advanced-hunting API are available. **Why required:** The entire measurement runs over advanced-hunting tables via Graph Security; P1 does not expose the full hunting surface this assessment reads. *Verify current plan-to-feature mapping and availability in your tenant.* **How to verify:** In `security.microsoft.com` → **Hunting** → **Advanced hunting**, run `DeviceNetworkEvents | take 1`. Rows return on P2. **How to implement if missing:** Assign Defender for Endpoint P2 (often bundled in M365 E5 / E5 Security) to the assessed devices' users/devices, then allow warm-up before measuring.

R2. Onboarded devices behind the SWG, with warmed-up telemetry

Description: A population of MDE-onboarded devices that actually sit **behind the loopback/forward SWG** and have been reporting for long enough to fill the hunting window. **Why required:** You can only measure obfuscation on devices whose traffic the proxy steers; freshly onboarded devices have no history to sample. **How to verify:** Run `MDEPROXY-Discovery.ps1` — it reports onboarded devices, `DeviceNetworkEvents` freshness, and which network tables are populated. Confirm the sample devices are the ones running the SWG agent. **How to implement if missing:** Onboard the devices (cross-reference the `MDEFULL` / `DEFXDR` onboarding solutions), confirm the SWG agent is active on them, and allow reporting warm-up (commonly a few days) before assessing.

R3. Security Reader (read-only) access to advanced hunting

Description: The **Security Reader** role (or higher) for the account running the scripts, with consent to the read-only Graph scopes. **Why required:** All measurement is read-only advanced hunting; no configuration role is needed to run the assessment. Keeping to Security Reader enforces the read-only posture. **How to verify:** Sign in and run `DeviceNetworkEvents | take 1` in Advanced hunting; run `MDEPROXY-Discovery.ps1` and confirm it connects with `ThreatHunting.Read.All`. **How to implement if missing:** Assign **Security Reader** in Entra → **Roles and administrators**; grant admin consent to the read scopes (`ThreatHunting.Read.All`, `SecurityEvents.Read.All`, `Organization.Read.All`) on first `Connect-MgGraph`.

R4. Identification of the SWG agent and its steering mode

Description: Know **which** SWG is in play (iboss, Zscaler Client Connector, generic PAC/loopback) and **how** it steers — route-based tunnel vs system-proxy/loopback, and whether specific destinations are excluded. **Why required:** The steering mode determines whether — and how much — destination telemetry is obscured, and it frames the remediation options. This is read from **your own** configuration; vendor internals are not asserted here. **How to verify:** From the SWG agent/console (read-only) and endpoint config: check the system proxy / PAC (`netsh winhttp show proxy`, the PAC URL, the agent's tunnel mode). Note the local listener address(es). **How to implement if missing:** Work with the network/SWG owner to document the current mode and the loopback/proxy address set. *Confirm the mode's exact behavior against current vendor documentation — this solution measures the effect rather than asserting the cause.*

R5. The loopback / proxy address set for destination classification

Description: The list of addresses the SWG uses locally — loopback (`127.0.0.0/8`, `::1`), any link-local/private proxy IP, and the agent's local listener IP/port — used to classify a `RemoteIP` as "obscured (proxy/loopback)" vs "true remote." **Why required:** The obfuscation measurement classifies

each connection's destination against this set; an incomplete set understates the impact. **How to verify:** Confirm the address set with the SWG owner and pass it to `MDEPROXY-Validate.ps1` (`-ProxyAddressSet`); the script echoes the set it used so the classification is reviewable. **How to implement if missing:** Derive the set from the PAC/system-proxy/agent config (R4). Default to loopback + RFC1918 as a conservative starting set, then refine with the SWG owner. *Document any address you include so the report is defensible.*

R6. Microsoft Graph PowerShell modules (Microsoft.Graph)

Description: The `Microsoft.Graph` PowerShell modules (Authentication + Security) on the workstation running the scripts. **Why required:** The `.ps1` scripts call `POST /security/runHuntingQuery` and read context via Graph. **How to verify:** `Get-Module -ListAvailable Microsoft.Graph.Authentication`. The scripts also install on first run. **How to implement if missing:** `Install-Module Microsoft.Graph -Scope CurrentUser` (the scripts do this automatically on first run if absent).

R7. A device sample definition (the assessment ring)

Description: A named, representative set of behind-the-SWG devices to measure first — the object `MDEPROXY Device Sample – Proxy Ring – Assess` used throughout the guide, scripts, and tests. **Why required:** The pattern is pilot-first: measure a representative ring, validate the findings, then expand across the estate. It also bounds the hunting queries. **How to verify:** Confirm the device list (by device name / device group / tag) exists and matches the SWG-steered population; `MDEPROXY-Discovery.ps1 -Sample` echoes it. **How to implement if missing:** Define the sample by an existing Defender device group or device tag (e.g. devices tagged for the SWG rollout), or supply an explicit device-name list to the scripts. Keep it representative of OS mix and site.

R8. A Microsoft support entitlement for the advisory case

Description: A support plan that lets you open a Microsoft **advisory (Severity C)** case for Defender for Endpoint. **Why required:** The authoritative answers (affected tables/fields, XFF behavior for endpoint telemetry, supported coexistence topology) come from Microsoft via the advisory case — this solution supplies the template, not the answers. **How to verify:** In `admin.microsoft.com` → **Support** → **New service request** (or the Defender portal's **Help** → **Support**), confirm you can open a case for the Defender/Endpoint product. **How to implement if missing:** Confirm the tenant's support entitlement (Unified/Premier/Professional). *Advisory (Sev C) is appropriate — this is a how-does-it-behave / architecture-review question, not a production outage.*

R9. (Optional) Log Analytics / SIEM export for longer-window analysis

Description: Where you need to look **beyond** the advanced-hunting retention window, a continuous export of the device tables to Log Analytics / your SIEM. **Why required:** Advanced-hunting retention is limited (*verify current retention*); a SIEM export lets you measure obfuscation over a longer period or historically. **How to verify:** Confirm a streaming API / continuous export of `DeviceNetworkEvents` exists and is queryable (cross-reference the `WSKQL` advanced-KQL solution for the query patterns). **How to implement if missing:** This is optional — the in-portal window is sufficient for the standard assessment. Configure continuous export only if longer-window analysis is required.

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures

This is a **read-only assessment**. Nothing here changes the SWG, the SWG agent, the endpoints, or the tenant. Follow the steps in order. Every object you create is a **document, sample, or case** — named with the convention `<Purpose> – <Scope> – <Stage>` and reused verbatim in later steps, the Testing Guide, and the scripts. Portal navigation paths are current as of authoring — *if a menu has moved, confirm the current path in the Defender portal.*

Objects you will create (names are fixed — reuse exactly):

- `MDEPROXY Device Sample – Proxy Ring – Assess` — the representative behind-the-SWG device set.
- `MDEPROXY Proxy Address Set – Tenant – Assess` — the loopback/proxy addresses used to classify destinations.
- `MDEPROXY Telemetry Baseline – Proxy Ring – Pilot` — the measured completeness/obfuscation baseline.
- `MDEPROXY Feature-Impact Matrix – Tenant – Validate` — features mapped to the proxy behavior.
- `MDEPROXY XFF Clarification – Tenant – Validate` — the X-Forwarded-For write-up.
- `MDEPROXY Remediation Plan – Tenant – Expand` — the supported-topology options for the change owners.

- `MDEPROXY Advisory Case – Microsoft – Expand` — the Severity C support ticket.
- `MDEPROXY Assessment Report – Tenant – Validate` — the compiled evidence package.

Stage 1 — Assess (environment + sample)

Step 1.1 — Confirm the read-only prerequisites

Confirm **R1–R7** from the Requirements Guide. In `security.microsoft.com` → **Hunting** → **Advanced hunting**, run `DeviceNetworkEvents | take 1` to confirm P2 + hunting access.

Step 1.2 — Run read-only discovery

On the assessment workstation:

```
powershell -ExecutionPolicy Bypass -File .\MDEPROXY-Discovery.ps1
```

It connects read-only (`ThreatHunting.Read.All`), reports the signed-in context and MDE plan hints, lists onboarded devices, checks `DeviceNetworkEvents` **freshness**, and reports **which network Advanced Hunting tables are populated** (`DeviceNetworkEvents`, `DeviceEvents`, `DeviceInfo`, `DeviceProcessEvents`). Confirm the tables are fresh before measuring.

Step 1.3 — Identify the SWG steering mode (read-only)

From your **own** endpoint/SWG configuration (not asserted here):

- Windows system proxy: `netsh winhttp show proxy` and the WinINET/PAC settings.
- The PAC URL / file, and whether it points traffic at a **local listener** (e.g. `127.0.0.1:<port>`).
- The SWG agent's **tunnel mode** (route-based tunnel vs system-proxy/loopback) and any **steering exclusions**, read from the agent/console.

Record the mode in `MDEPROXY Remediation Plan – Tenant – Expand` (Stage 5). *Confirm the mode's exact behavior against current vendor documentation — this assessment measures the effect, not the cause.*

Step 1.4 — Define the device sample

Create `MDEPROXY Device Sample – Proxy Ring – Assess` — a representative set of behind-the-SWG devices (by Defender device group, device tag, or an explicit device-name list), covering your OS mix and sites. Keep it small enough to reason about, representative enough to trust.

Step 1.5 — Assemble the proxy address set

Create `MDEPROXY Proxy Address Set – Tenant – Assess` — the loopback/proxy addresses (from Step 1.3): loopback (`127.0.0.0/8`, `::1`), the agent's local listener IP(s), and any private proxy IP. This set classifies a connection's `RemoteIP` as **obscured (proxy/loopback)** vs **true remote**. Document every address you include so the classification is defensible.

Stage 2 — Configure (measurement parameters)

Step 2.1 — Set the measurement window and thresholds

Decide the lookback window (default 7 days, within the hunting retention — *verify current retention*) and the **obfuscation threshold** that constitutes a finding (e.g. "≥ X% of outbound connections resolve to the proxy address set"). Record these in the report header.

Step 2.2 — Stage the KQL query pack

Open `MDEPROXY-Hunt.kql` in **Advanced hunting**. It contains the read-only measurement queries used in Stage 3 (destination-obfuscation share, `DeviceNetworkEvents` completeness, table population, and network-dependent detection coverage). Confirm each query parses against your tenant's schema; adjust table/column names only if your schema differs.

Stage 3 — Pilot (measure the sample, read-only)

Step 3.1 — Measure destination obfuscation

Run:

```
powershell -ExecutionPolicy Bypass -File .\MDEPROXY-Validate.ps1 `
  -Sample "MDEPROXY Device Sample – Proxy Ring – Assess" `
  -ProxyAddressSet @('127.0.0.0/8','::1','<local-listener-ip>') `
  -Days 7
```

It measures, over the sample and window: the **share of outbound** `DeviceNetworkEvents` **whose** `RemoteIP` **falls in the proxy address set** (obscured) vs a true remote, the count of connections with **missing/empty** `RemoteUrl`, and the per-device distribution. It echoes the address set it used and writes `MDEPROXY Telemetry Baseline – Proxy Ring – Pilot`. **Read-only.**

Step 3.2 — Measure `DeviceNetworkEvents` *completeness and table population*

Still read-only, `MDEPROXY-Validate.ps1 -Check Completeness` (and the `MDEPROXY-Hunt.kql` completeness queries) report: whether `DeviceNetworkEvents` is populated for each sample device, the ratio of network events to process events (a low ratio can indicate suppressed/flattened network telemetry), and which of the network tables return rows. Record gaps against the baseline.

Step 3.3 — Enumerate network-dependent features and detections

Run:

```
powershell -ExecutionPolicy Bypass -File .\MDEPROXY-FeatureImpact.ps1 `
  -Sample "MDEPROXY Device Sample – Proxy Ring – Assess"
```

It enumerates, read-only: **custom detection rules** and **hunting queries** that reference `RemoteIP` / `RemoteUrl` / `RemotePort` (directly exposed to obfuscation), and the **custom TI indicators** of type **IP / URL / domain** (whose endpoint-layer matching depends on true-remote visibility). It also lists the portal-only checks for **Network Protection** and **SmartScreen** state. Output seeds the feature-impact matrix.

Stage 4 — Validate (findings, matrix, XFF)

Step 4.1 — Build the feature-impact matrix

Create `MDEPROXY Feature-Impact Matrix – Tenant – Validate` from the Stage 3 output. Use this structure — the **"Confirm with Microsoft"** column is mandatory and must not be pre-answered:

Feature / signal	Depends on true-remote destination?	Observed effect in this tenant (from measurement)	Confirm with Microsoft
`DeviceNetworkEvents` (`RemoteIP`/`RemoteUrl`/`RemotePort`)	Yes — directly	**Measured**: % obscured to proxy/loopback; missing `RemoteUrl` count	Confirm authoritative `RemoteIP`/`RemoteUrl` semantics when a local proxy is present
Hunts / custom detections pivoting on remote destination	Yes — inherit the above	List of affected rules/queries (from `MDEPROXY-FeatureImpact.ps1`)	Confirm which built-in detections read endpoint remote-destination
SmartScreen (URL/download reputation)	Reputation on destination	Configuration state noted (portal check)	**Confirm** whether/how loopback-forward steering affects SmartScreen evaluation
Network Protection (IP/URL/domain block, web content filtering, custom indicators)	Yes — evaluates destination	Configuration state + indicator inventory noted	**Confirm** whether NP evaluates the true remote or the loopback destination under this topology
Threat-intelligence indicator matching (IP/URL/domain)	Yes — endpoint-layer match	Count of IP/URL/domain indicators that may not match	**Confirm** endpoint-layer TI matching behavior with a local proxy

Fill the **"Observed effect"** column from your measurement; leave **"Confirm with Microsoft"** as the open question carried into the advisory case. **Do not invent Microsoft's answer.**

Step 4.2 — Author the XFF clarification

Create `MDEPROXY XFF Clarification – Tenant – Validate` using the section below verbatim as its basis. Keep the confirm-with-Microsoft framing intact.

> ***X-Forwarded-For (XFF) — what it is, and what it does / does not restore for endpoint telemetry***
>
> ***What XFF is.*** `X-Forwarded-For` *is an* ***HTTP request header*** *added by a forward or reverse proxy so that the* ***destination server*** *can learn the* ***original client's IP address*** *even though the connection reaching it comes from the proxy. It is a* ***server-side*** *attribution mechanism: it flows *forward*, from proxy toward the destination, and it is read by the* ***destination server, its load balancer, WAF, or application logs*** *— and by the SWG's own logs.* ***The direction that matters here.*** *The telemetry problem in this solution is on the* ***endpoint*** *side: the MDE sensor records the connection the *application* made — to the* ***local loopback listener*** *— so the endpoint's* `DeviceNetworkEvents` *shows the proxy/loopback as* `RemoteIP` *and often has no true* `RemoteUrl`*. XFF addresses the* ***opposite*** *direction of the flow (letting the *far* server identify the *client*); it is carried* ***inside the request the SWG sends onward****, not injected back into the endpoint's own network-stack records.*
>
> ***What XFF therefore does.*** *It restores* ***client-IP visibility at the destination and in server-side logs***

*— useful for the SWG's records, the destination's logs, and any server-side investigation. It is genuinely valuable, but for **server-side** attribution. **What XFF does not (appear to) do for endpoint telemetry.** Because XFF is a header in the outbound request rather than a feedback path into the endpoint sensor, it is **not expected to repopulate the true remote destination into the endpoint's** `DeviceNetworkEvents` (`RemoteIP` / `RemoteUrl`). In other words, XFF helps the *server* know the *client*; it does not, on its face, help the *endpoint* record the *true server*. **This is our reasoning about the mechanism, not Microsoft's definitive statement — confirm the current behavior with Microsoft** (it is one of the explicit asks in the advisory case, Step 5.3). Do **not** present this as Microsoft's confirmed position until the case returns.*

Step 4.3 — Compile the assessment report

Run:

```
powershell -ExecutionPolicy Bypass -File .\MDEPROXY-Report.ps1 `
  -BaselinePath ".\MDEPROXY-telemetry-baseline.json"
```

It compiles `MDEPROXY Assessment Report – Tenant – Validate` — the measurement summary (obfuscation %, completeness, table population), the feature-impact matrix, the XFF clarification, and the open questions for Microsoft. **Read-only; it only reads the saved measurement outputs.**

Stage 5 — Expand (estate assessment + remediation plan + advisory case)

Step 5.1 — Expand the measurement across the estate

Re-run `MDEPROXY-Validate.ps1` and `MDEPROXY-FeatureImpact.ps1` against a broader device set (or your continuous-export/SIEM copy for a longer window, R9). Compare against `MDEPROXY Telemetry Baseline – Proxy Ring – Pilot` to confirm the sample was representative.

Step 5.2 — Draft the supported-topology remediation (plan only)

Create `MDEPROXY Remediation Plan – Tenant – Expand` as a set of **options for the customer's network/security/architecture owners** — this package does **not** apply any of them. **Flag every option "verify against current Microsoft guidance"** and record the security tradeoff.

Options to document (verify each against current Microsoft and vendor guidance — do not assert):

1. **Steering mode.** Evaluate whether the SWG can run in a **route-based / tunnel mode** where the endpoint's network stack still egresses to the **true remote** (better endpoint telemetry) instead of a **system-proxy / loopback** mode that flattens destinations to the local listener. *Confirm the exact per-mode behavior with the vendor and Microsoft.*

2. **Steering exclusions.** Where telemetry fidelity matters most, evaluate **excluding specific destinations from local steering** so their true remote is visible to the sensor — weighed against the SWG filtering coverage lost. **Security tradeoff — customer decision.**

3. **Service-endpoint exclusions.** Ensure the **Defender/MDE service URLs** are not broken by SWG interception/decryption (this is connectivity, cross-reference `NETDX`) — separate from, but adjacent to, the telemetry question.

4. **Connector / integration deployment.** Evaluate the vendor's documented **MDE integration/connector** and any supported deployment option that preserves endpoint destination visibility. *Vendor internals are vendor-specific; confirm the supported option with Microsoft and the

vendor.*

5. **Compensating visibility.** Where a mode change is not acceptable, document **compensating controls** — e.g. correlating the SWG's own destination logs with endpoint telemetry in the SIEM (cross-reference `WSKQL` / `DEFXDR`) — as an interim measure, clearly labeled as a workaround, not a Microsoft-supported restoration.

Present these as options with tradeoffs; the **choice and the change belong to the customer's owners**.

Step 5.3 — Open the Microsoft advisory case

Create `MDEPROXY Advisory Case – Microsoft – Expand` in `admin.microsoft.com` → **Support** → **New service request** (or Defender portal **Help** → **Support**), product **Microsoft Defender for Endpoint**, **Severity C (advisory)**. Use the template below verbatim, attaching `MDEPROXY Assessment Report – Tenant – Validate`.

> ***Title:*** *Advisory — Endpoint network telemetry (DeviceNetworkEvents) obfuscation with a loopback/forward SWG proxy: confirm affected tables, feature impact, XFF applicability, and supported coexistence topology* ***Severity:*** *C (advisory / how-does-it-behave; no production outage)*
> ***Environment:*** *Defender for Endpoint P2. Endpoints run a secure web gateway agent (`<iboss / Zscaler Client Connector / generic PAC-loopback>`) that steers outbound traffic through a* ***local loopback listener*** *before egress. Attached assessment (`MDEPROXY Assessment Report – Tenant – Validate`) shows a measured <X>% of outbound `DeviceNetworkEvents` resolving to loopback/proxy addresses rather than true remote destinations, and <N> connections with missing `RemoteUrl`.* ***We are requesting Microsoft to confirm (do not want us to assume):*** *1.* ***Validate telemetry impact*** *— Confirm which Advanced Hunting tables and fields (`DeviceNetworkEvents.RemoteIP` / `RemoteUrl` / `RemotePort`, related `DeviceEvents`) are affected when endpoint traffic is steered through a local loopback proxy before egress, and whether this is expected behavior. 2.* ***Document feature impact*** *— Confirm the impact, under this topology, on* ***SmartScreen***, ***Network Protection*** *(including web content filtering and custom IP/URL/domain indicators), and* ***threat-intelligence indicator matching*** *at the endpoint layer. Does Network Protection / TI matching evaluate the* ***true remote*** *or the* ***loopback*** *destination? 3.* ***XFF applicability*** *— Confirm whether* ***X-Forwarded-For*** *(or any equivalent mechanism / SWG integration) restores the* ***true remote destination into endpoint*** `DeviceNetworkEvents`*, or whether XFF only aids* ***server-side*** *client attribution and does* ***not*** *repopulate endpoint telemetry. 4.* ***Architecture review*** *— Confirm the* ***supported MDE + SWG coexistence topology****: which SWG steering mode(s) / exclusions / connector options are supported and preserve endpoint destination telemetry, and any Microsoft guidance on the tradeoffs. 5.* ***Advanced-hunting schema*** *— Confirm the* ***authoritative semantics*** *of `RemoteIP` / `RemoteUrl` (and how to distinguish a proxied connection from a direct one in the schema) when a local proxy is present, so our hunts and detections can be written correctly.*
> ***Attachments:*** `MDEPROXY Assessment Report – Tenant – Validate` *(measurement, feature-impact matrix, XFF reasoning, sample scope).*

Step 5.4 — Record and hand off

Record the case number on `MDEPROXY Advisory Case – Microsoft – Expand`, attach Microsoft's response to `MDEPROXY Assessment Report – Tenant – Validate` when it returns, and **update the feature-impact matrix's "Confirm with Microsoft" column and the XFF clarification with Microsoft's authoritative answer** — replacing the open questions with confirmed behavior. Hand `MDEPROXY Remediation Plan – Tenant – Expand` to the network/security/architecture owners for the change decision.

Read-only guarantee. No step above modifies the SWG, the agent, the endpoints, or the tenant. The only artifacts produced are documents, a measured baseline, a report, and a support case. Any topology change is executed by the customer's own change process, outside this package.

5. Testing Guide — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures

Because this solution is a **read-only assessment**, "testing" means confirming the **measurement is trustworthy and the deliverables are complete and honestly framed** — not that a control was enforced. Every test has a **pass/fail gate**; do not present the assessment as conclusive until the gates pass. Objects are referenced by the exact names from the Implementation Guide; scripts by name.

Part A — Assessment readiness

A-Test 1 — Read-only access and tables available

Procedure: Run `MDEPROXY-Discovery.ps1`. Confirm it connects with `ThreatHunting.Read.All`, `DeviceNetworkEvents | take 1` returns rows (P2), and the network tables (`DeviceNetworkEvents`, `DeviceEvents`, `DeviceInfo`, `DeviceProcessEvents`) are populated and fresh. **Pass:** connects read-only; P2 confirmed; network tables fresh. **Fail:** no rows / P1 only / stale — see Troubleshooting *No DeviceNetworkEvents data* and confirm R1/R2.

A-Test 2 — Sample is representative and behind the SWG

Procedure: Confirm `MDEPROXY Device Sample – Proxy Ring – Assess` covers the OS mix and sites, and that each sample device actually runs the SWG agent (Step 1.3/1.4). **Pass:** the sample matches the behind-the-SWG population. **Fail:** sample includes non-steered devices or misses a platform — redefine per Step 1.4.

A-Test 3 — Proxy address set is complete and documented

Procedure: Confirm `MDEPROXY Proxy Address Set – Tenant – Assess` lists the loopback, local-listener, and proxy addresses derived from the actual config (R4/R5); `MDEPROXY-Validate.ps1` echoes the set it used. **Pass:** every classifying address is documented and justified. **Fail:** the set is a guess or the script echoes a different set — rebuild from the real PAC/system-proxy/agent config.

Part B — Measurement validity

B-Test 1 — Destination-obfuscation measured

Procedure: Run `MDEPROXY-Validate.ps1 -Sample "MDEPROXY Device Sample – Proxy Ring – Assess" -ProxyAddressSet <set> -Days 7`. Confirm `MDEPROXY Telemetry Baseline – Proxy Ring – Pilot` records the **% of outbound** `DeviceNetworkEvents` **whose** `RemoteIP` **is in the proxy address set**, the missing-`RemoteUrl` count, and the per-device distribution. **Pass:** a numeric obfuscation share and completeness figure are recorded for the sample. **Fail:** empty/zero across all devices — confirm the address set (A-Test 3) and window; a true 0% means no obfuscation (a valid,

reportable finding).

B-Test 2 — Completeness and table population measured

Procedure: Run `MDEPROXY-Validate.ps1 -Check Completeness` and the `MDEPROXY-Hunt.kql` completeness queries. Confirm the network-to-process event ratio and per-table population are recorded per device. **Pass:** completeness metrics recorded; any empty table flagged. **Fail:** no completeness data — re-run; if a table is empty, note it as a finding, not a script error.

B-Test 3 — Feature/detection enumeration produced

Procedure: Run `MDEPROXY-FeatureImpact.ps1`. Confirm it lists custom detections/hunts referencing `RemoteIP`/`RemoteUrl`/`RemotePort` and the IP/URL/domain TI indicators, plus the portal-only NP/SmartScreen checks. **Pass:** a concrete list of exposed rules and indicators is produced (or an explicit "none found"). **Fail:** the enumeration errors out — correct the query/scope and re-run.

B-Test 4 — Results are reproducible

Procedure: Re-run `MDEPROXY-Validate.ps1` with the same parameters. Confirm the obfuscation share is stable within a small tolerance (traffic varies; the order of magnitude should hold). **Pass:** consecutive runs agree in magnitude. **Fail:** wild swings — check the window is long enough and the sample is stable.

Part C — Deliverable integrity (honest framing)

C-Test 1 — Feature-impact matrix keeps the "Confirm with Microsoft" column open

Procedure: Open `MDEPROXY Feature-Impact Matrix – Tenant – Validate`. Confirm the **"Observed effect"** column is filled from measurement and the **"Confirm with Microsoft"** column states the open question — **not** an invented Microsoft answer. **Pass:** measured effects present; Microsoft-authoritative items remain open questions. **Fail:** any cell asserts Microsoft's definitive position without the case — remove the assertion; it is not yet confirmed.

C-Test 2 — XFF clarification is correctly framed

Procedure: Read `MDEPROXY XFF Clarification – Tenant – Validate`. Confirm it explains XFF as **server-side**, reasons about endpoint telemetry, and **explicitly defers to Microsoft** ("confirm current behavior with Microsoft"). **Pass:** XFF explained as server-side; endpoint conclusion framed as our reasoning pending Microsoft. **Fail:** it asserts Microsoft's answer — re-frame per Step 4.2.

C-Test 3 — Remediation is a plan of options, not an applied change

Procedure: Read `MDEPROXY Remediation Plan – Tenant – Expand`. Confirm every option is flagged "verify against current Microsoft guidance," records the security tradeoff, and is addressed to the customer's owners — and that **nothing was applied** by this package. **Pass:** options only, verify-flagged, tradeoffs noted, no change applied. **Fail:** an option reads as done or asserts support without the verify flag — correct it.

C-Test 4 — Advisory case template complete

Procedure: Open `MDEPROXY Advisory Case – Microsoft – Expand`. Confirm it is Severity C and includes all five asks (validate-telemetry, document-feature-impact, XFF, architecture-review, advanced-hunting-schema) with the assessment report attached. **Pass:** all five asks present; Sev C; report attached. **Fail:** an ask missing or wrong severity — complete per Step 5.3.

C-Test 5 — Report compiled and read-only guarantee intact

Procedure: Run `MDEPROXY-Report.ps1`. Confirm `MDEPROXY Assessment Report – Tenant – Validate` contains the measurement summary, matrix, XFF section, and open questions — and that no script or step changed the proxy or tenant. **Pass:** report complete; read-only guarantee holds. **Fail:** missing sections — re-run the relevant validate step first.

Assessment exit gate: all of Part A and Part B pass (trustworthy measurement) and all of Part C passes (honestly framed, Microsoft-deferred deliverables) before the assessment is presented as conclusive or the advisory case is closed against. Re-run Parts A–B after any SWG mode change, connector upgrade, or once Microsoft's advisory answer returns, to re-measure the delta.

6. Troubleshooting — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures

Symptom → cause → fix for the common failures. This is a **read-only assessment**, so most "failures" are about **trusting the measurement** or **framing the deliverables**, not about a broken control. Nothing here changes the proxy or tenant.

No DeviceNetworkEvents data (can't measure obfuscation at all)

Symptom: `MDEPROXY-Discovery.ps1` shows `DeviceNetworkEvents` = 0 rows, or `DeviceNetworkEvents | take 1` returns nothing. **Likely causes:** Defender for Endpoint **P1** (advanced hunting / full network table not available); devices not onboarded or not yet warmed up; the account lacks `ThreatHunting.Read.All`; wrong Graph cloud environment. **Fix:** Confirm **P2** (R1) and re-check the plan hints in discovery. Confirm the sample devices are onboarded and have reported for a few days (R2). Confirm the account is **Security Reader** with the read scope consented (R3). For GCC/DoD, pass `-Environment USGov/USGovDoD`. A genuinely empty table on P2 with onboarded, warmed-up devices is itself a finding — escalate as advisory ask #1.

Obscured share comes back 0% but you expected obfuscation

Symptom: `MDEPROXY-Validate.ps1` reports 0% obscured though the SWG is clearly in loopback mode. **Likely causes:** the **proxy address set is incomplete** — the real local listener IP isn't in it; the SWG is actually in a **route-based tunnel** mode (which *does* preserve true remote — a good result); or the window/sample missed the steered traffic. **Fix:** Run `MDEPROXY-Hunt.kql` **Q3 (top RemoteIP values)** to reveal the actual local listener/proxy addresses, then add them to `-ProxyAddressSet` (R5) and re-measure. Confirm the steering mode (Step 1.3) — a true route-based tunnel legitimately shows low obfuscation; report that as the finding, not a failure. Widen the window or sample if traffic was sparse.

Obscured share looks implausibly high (≈100% on everything)

Symptom: Every device and process shows ~100% obscured. **Likely causes:** the address set is **too broad** (e.g. it includes an RFC1918 range that also covers legitimate internal destinations), so real internal traffic is misclassified as "proxy." **Fix:** Tighten `MDEPROXY Proxy Address Set – Tenant – Assess` to only the loopback + the confirmed local-listener/proxy IPs; do not include broad private ranges unless they truly are the proxy. Re-run `MDEPROXY-Hunt.kql` Q3 to confirm the top destinations really are the

proxy before trusting the number.

RemoteUrl is missing but RemoteIP looks like a real remote

Symptom: High missing-`RemoteUrl` count, yet `RemoteIP` is not in the proxy set. **Likely causes:** URL is only recorded for certain connection types/protocols; TLS without SNI capture; or the sensor recorded the connection without a resolvable URL — not necessarily proxy-related. **Fix:** Treat missing-`RemoteUrl` as a **secondary** indicator, not proof of obfuscation; the primary metric is the `RemoteIP`-in-proxy-set share. Note the missing-URL figure in the report and add the URL-visibility question to advisory ask #5 (schema semantics).

Completeness ratio is low but obfuscation is low too

Symptom: `Net/Proc` ratio is very low, yet the obscured share is small. **Likely causes:** low network activity on those devices, aggressive dedup, or a warm-up gap — a low ratio alone does not prove flattened telemetry. **Fix:** Corroborate with the obscured share and Q3 before concluding. Report the ratio as context, not as a standalone finding. Re-measure over a longer window if the sample is quiet.

Feature enumeration returns nothing (indicators / detections empty)

Symptom: `MDEPROXY-FeatureImpact.ps1` lists no indicators or no detection rules. **Likely causes:** the API path differs in your cloud (beta vs v1.0), the account lacks the indicator/detection read scope, or there genuinely are none configured. **Fix:** The script tries both beta and v1.0 paths; if both fail, enumerate manually — **Settings** → **Endpoints** → **Indicators** (IP/URL/Domain) and **Hunting** → **Custom detection rules** (any KQL using `RemoteIP`/`RemoteUrl`/`RemotePort`). *Verify the current API path.* "None found" is a legitimate result — record it.

The matrix or report asserts a Microsoft answer

Symptom: A reviewer flags that the feature-impact matrix or XFF section states Microsoft's definitive behavior (C-Test 1/C-Test 2 fail). **Cause:** an open question was filled with an assumption before the advisory case returned. **Fix:** Restore the **"Confirm with Microsoft"** cell to an open question; re-frame the XFF conclusion as *our reasoning, pending Microsoft*. Only replace the open items after `MDEPROXY Advisory Case – Microsoft – Expand` returns an authoritative answer, and cite the case number when you do.

The remediation reads as an applied change

Symptom: `MDEPROXY Remediation Plan – Tenant – Expand` sounds like steps were executed (C-Test 3 fails). **Cause:** options were written imperatively instead of as choices for the owners. **Fix:** Re-write each option as an evaluate/verify choice with its **security tradeoff**, addressed to the network/security/architecture owners, flagged "verify against current Microsoft guidance." Confirm nothing was changed by this package — it never is.

SWG steering mode can't be confirmed

Symptom: You can't tell whether the agent is route-based tunnel vs system-proxy/loopback (Step 1.3). **Cause:** the mode lives in the SWG console/agent config, which is vendor-specific. **Fix:** Read it from

`netsh winhttp show proxy`, the PAC file, and the agent's own settings with the SWG owner; confirm the exact behavior against **current vendor documentation** (do not assert it). Regardless of the labelled mode, the **measured** obscured share (Q1) is the ground truth for the report.

Sensor can reach the service, but you're still worried about connectivity

Symptom: You suspect the proxy is *also* blocking the sensor's own reporting. **Cause:** that is a **connectivity** problem, not this telemetry-visibility assessment. **Fix:** Use the **"Endpoint Network Connectivity Troubleshooting Toolkit"** (`NETDX`) for sensor-to-service reachability and the `NSSWG` line for SWG exclusions of the Defender service URLs. This solution assumes the sensor reports and focuses on **destination telemetry fidelity**.

Advisory case gets bounced to a higher/lower severity

Symptom: Microsoft re-classifies the ticket. **Cause:** Severity C (advisory) is correct for a how-does-it-behave / architecture-review question with no outage; a reclassification usually means the framing implied an incident. **Fix:** Keep the five asks framed as **confirmation/architecture** questions (Step 5.3), attach `MDEPROXY Assessment Report – Tenant – Validate`, and keep it Severity C. If a genuine outage exists (sensor not reporting), that is a separate, higher-severity connectivity case.

7. Best Practices — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures

Keep the assessment trustworthy, honestly framed, and reusable.

Keep it read-only and keep it that way

- **Run as Security Reader.** The whole assessment fits inside a read-only role; do not run it as a configuration admin "just in case." Least privilege *is* the read-only guarantee.
- **Never let the assessment become a change.** This package measures; it does not touch the SWG, the agent, the endpoints, or the tenant. If someone wants to change steering mode, that is the customer's change process, not a step here.
- **Version the read scopes.** Consent only to `ThreatHunting.Read.All`, `SecurityEvents.Read.All`, and `Organization.Read.All`. If a script ever needs more, question why.

Make the measurement defensible

- **Document the proxy address set.** Every address that classifies a destination as "obscured" must be justified from the real config. An unexplained address in the set invalidates the finding.
- **Report a true 0% as a real result.** If a device or mode shows no obfuscation, that is a valid, valuable finding — not a failed run. Distinguish "no obfuscation" from "no data."
- **Mind the retention window.** Advanced hunting keeps a limited window (*verify current retention*). For historical or long-window analysis, use the SIEM/continuous-export copy (R9, cross-reference `WSKQL`).

• **Sample representatively, then confirm on expansion.** Pilot on `MDEPROXY Device Sample – Proxy Ring – Assess`, then confirm the sample was representative when you expand across the estate.

Never invent Microsoft's answers

• **Keep the "Confirm with Microsoft" column open** until the advisory case returns. The measured obfuscation is yours to assert; the *feature behavior*, *XFF applicability*, *affected-table list*, and *supported topology* are **Microsoft's to confirm**.

• **Frame XFF as reasoning, not ruling.** XFF is server-side; our endpoint conclusion is well-reasoned but stays "confirm with Microsoft" until the case answers it.

• **Flag every remediation option "verify against current guidance."** SWG mode behavior is vendor- and version-specific; supported coexistence is Microsoft's determination. Present options and tradeoffs; the customer's owners decide.

Defer the decisions that aren't technical

• **The steering tradeoff is the customer's.** Preserving endpoint telemetry may mean changing how the SWG steers or excludes traffic — trading filtering coverage for visibility. That is a **security/network/architecture decision**, deferred to those owners; the guide supplies the evidence, not the ruling.

• **Compliance and records/legal determinations** stay with the customer's compliance/legal functions.

Operate the loop

• **Re-run after any change.** SWG mode change, connector upgrade, PAC change, or Microsoft's advisory answer — re-run Parts A–B to re-measure the delta. The assessment is a reusable instrument, not a one-shot.

• **Fold Microsoft's answer back in.** When the case returns, replace the open questions in the matrix and XFF section with the confirmed behavior, and note the case number for provenance.

Where to go next (adjacent catalog areas)

• **Endpoint Network Connectivity Troubleshooting Toolkit** (`NETDX`) — if the question is "can the sensor *reach* the service through the proxy" (connectivity), not "is the *destination telemetry* obscured" (this solution).

• **Netskope SWG line** (`NSSWG` deployment, `NSCASB` CASB, SSL/TLS inspection, traffic steering, and the SASE/SSE blueprint) — for **deploying and operating** the gateway itself, including steering modes and exclusions.

• **Microsoft Defender for Endpoint** (`MDEFULL`) — the comprehensive MDE stand-up, tuning, enablement, and PoC.

• **Microsoft Defender XDR — Unified Onboarding** (`DEFXDR`) — cross-pillar correlation, and the place to fuse SWG destination logs with endpoint telemetry as a compensating control.

• **Advanced KQL for SecOps** (`WSKQL`) — the KQL patterns for longer-window, SIEM-side analysis of the device tables.

8. Deliverables Summary — Defender for Endpoint Network Telemetry in Web Proxy / SWG Architectures

Everything included in this solution, and how it fits together.

Documentation (source guides)

• **Executive Summary** — what the assessment is, the outcome (tenant-specific evidence + supported-topology plan + Microsoft advisory case), the boundary, and effort.

• **01 · Solution Overview** — how loopback/forward-proxy steering obscures endpoint destination telemetry, where in the Defender stack it matters, scope, out-of-scope (with cross-references), and the connections/auth used.

• **02 · Requirements** — the solution-specific prerequisites (P2 + advanced hunting, behind-the-SWG onboarded devices, Security Reader read-only access, SWG-mode identification, the proxy address set, Graph modules, the device sample, a support entitlement, optional SIEM export). Category-level prerequisites are in the separate **Global Requirements — Defender** file.

• **03 · Implementation** — the exact, ordered, **read-only** steps (Assess → Configure → Pilot → Validate → Expand), every object named `<Purpose> – <Scope> – <Stage>`, plus the **XFF clarification** text, the **feature-impact matrix** structure, the **supported-topology remediation** options, and the **Microsoft advisory ticket template**.

• **05 · Testing** — pass/fail gates for measurement validity and honest deliverable framing (including "keep the Confirm-with-Microsoft column open" and "XFF is reasoning, not ruling").

• **06 · Troubleshooting** — symptom → cause → fix for the common visibility-gap and measurement failures.

• **07 · Best Practices** — keep-it-read-only, make-the-measurement-defensible, never-invent-Microsoft's-answers, defer-the-non-technical-decisions, operate-the-loop, and where to go next.

Scripts (read-only assessment + reporting)

All scripts are **read-only**: they sample, measure, and report. **None** changes the SWG, the agent, the endpoints, or the tenant.

• `MDEPROXY-Discovery.ps1` — read-only readiness: signed-in context, MDE plan (P2) hints, onboarded devices, `DeviceNetworkEvents` freshness, and **which network Advanced Hunting tables are populated**. Confirms the environment is ready to measure.

• `MDEPROXY-Validate.ps1` — the core measurement: over the device sample and window, the **share of outbound** `DeviceNetworkEvents` **whose** `RemoteIP` **is a loopback/proxy address vs a true remote** (destination obfuscation), the missing-`RemoteUrl` count, and `DeviceNetworkEvents` completeness. Writes the telemetry baseline.

• `MDEPROXY-FeatureImpact.ps1` — enumerates the network-dependent surface: custom detections/hunts referencing `RemoteIP`/`RemoteUrl`/`RemotePort`, the IP/URL/domain TI indicators exposed to obfuscation, and the portal-only Network Protection / SmartScreen checks. Seeds the feature-impact matrix.

• `MDEPROXY-Hunt.kql` — the Advanced Hunting KQL query pack: destination-obfuscation share, `DeviceNetworkEvents` completeness, network-table population, and network-dependent detection coverage — run interactively in the portal or via the scripts.

• `MDEPROXY-Report.ps1` — compiles the **assessment report** from the saved measurement outputs: obfuscation/completeness summary, feature-impact matrix, XFF clarification, and the open questions for the Microsoft advisory case.

Named objects produced (documents, not changes)

`MDEPROXY Device Sample – Proxy Ring – Assess` · `MDEPROXY Proxy Address Set – Tenant – Assess` · `MDEPROXY Telemetry Baseline – Proxy Ring – Pilot` · `MDEPROXY Feature-Impact Matrix – Tenant – Validate` · `MDEPROXY XFF Clarification – Tenant – Validate` · `MDEPROXY Remediation Plan – Tenant – Expand` · `MDEPROXY Advisory Case – Microsoft – Expand` · `MDEPROXY Assessment Report – Tenant – Validate`.

How it is delivered

Delivered as a **Complete Solution Package (PDF + Word + scripts)** — the documentation as a Word document and PDF, plus all five scripts (four PowerShell + one KQL pack) in a single ZIP, yours to keep and reuse on every re-assessment.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MDEPROXY-Discovery.ps1

Purpose: Confirms the environment is ready to MEASURE proxy-induced network-telemetry obfuscation. Reports the signed-in context, indicative MDE plan (P1/P2) hints, the onboarded-device count, DeviceNetworkEvents freshness, and WHICH network Advanced Hunting tables are populated (DeviceNetworkEvents, DeviceEvents, DeviceInfo, DeviceProcessEvents). Lists the read-only portal/SWG checks to perform manually. Makes NO changes to the proxy, agent, or tenant. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDEPROXY-Discovery.ps1 Installs the Microsoft.Graph modules on first run. Connects read-only with Connect-MgGraph (scope ThreatHunting.Read.All + context scopes). Government cloud: pass -Environment. EXPECTED RESULTS A readiness snapshot (JSON) — tenant/context, plan hints, device count, table freshness, and the manual checks — confirming you can proceed to MDEPROXY-Validate.ps1. Read-only throughout. .NOTES White-label. Read-only. Reports on configuration/telemetry only; changes nothing. Licensing/plan facts are indicative — verify current P1/P2 availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDEPROXY-discovery.json", [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global', [switch]$Sample)

How to run: powershell -ExecutionPolicy Bypass -File .\MDEPROXY-Discovery.ps1 Installs the Microsoft.Graph modules on first run. Connects read-only with Connect-MgGraph (scope ThreatHunting.Read.All + context scopes). Government cloud: pass -Environment. EXPECTED RESULTS A readiness snapshot (JSON) — tenant/context, plan hints, device count, table freshness, and the manual checks — confirming you can proceed to MDEPROXY-Validate.ps1. Read-only throughout. .NOTES White-label. Read-only. Reports on configuration/telemetry only; changes nothing. Licensing/plan facts are indicative — verify current P1/P2 availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDEPROXY-discovery.json", [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global', [switch]$Sample)

MDEPROXY-FeatureImpact.ps1

Purpose: Seeds the feature-impact matrix. Read-only, it enumerates: custom detection rules whose query references RemoteIP / RemoteUrl / RemotePort (directly exposed to obfuscation), and the custom threat-intelligence indicators of type IP / URL / Domain (whose endpoint-layer matching depends on true-remote visibility). It also prints the portal-only checks for Network Protection and SmartScreen state. It does NOT assert Microsoft's feature behavior — those cells stay 'confirm with Microsoft' in the matrix. Changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDEPROXY-FeatureImpact.ps1 Needs ThreatHunting.Read.All (+ ThreatIndicators.Read.All / relevant read scope for indicators). Some inventories may require the Defender/Graph Security beta endpoint — verify current API. Government cloud: pass -Environment. EXPECTED RESULTS MDEPROXY-feature-impact.json — the list of exposed custom detections/hunts and IP/URL/domain indicators, plus the manual NP/SmartScreen checks, to paste into the feature-impact matrix. .NOTES White-label. Read-only. Enumerates configuration only. Microsoft-authoritative feature behavior is obtained via the advisory case, not asserted here. Verify current indicator/detection API paths. > [CmdletBinding()] param([string[]]$Sample, [string]$OutputPath = ".\MDEPROXY-feature-impact.json", [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\MDEPROXY-FeatureImpact.ps1 Needs ThreatHunting.Read.All (+ ThreatIndicators.Read.All / relevant read scope for indicators). Some inventories may require the Defender/Graph Security beta endpoint — verify current API. Government cloud: pass -Environment. EXPECTED RESULTS MDEPROXY-feature-impact.json — the list of exposed custom detections/hunts and IP/URL/domain indicators, plus the manual NP/SmartScreen checks, to paste into the feature-impact matrix. .NOTES White-label. Read-only. Enumerates configuration only. Microsoft-authoritative feature behavior is obtained via the advisory case, not asserted here. Verify current indicator/detection API paths. > [CmdletBinding()] param([string[]]$Sample, [string]$OutputPath = ".\MDEPROXY-feature-impact.json", [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

MDEPROXY-Hunt.kql

Purpose: // Interactive KQL to MEASURE how much endpoint destination telemetry the SWG proxy obscures, // check DeviceNetworkEvents completeness/table population, and surface the network-dependent // detection surface. Read-only — run in security.microsoft.com > Hunting > Advanced hunting, // or via MDEPROXY-Validate.ps1 / MDEPROXY-FeatureImpact.ps1. // // HOW TO RUN // Set the two parameters at the top of each query (device sample + proxy address set), then run // the query you want. Nothing here changes the proxy, the agent, or the tenant. // // EXPECTED RESULTS // Per-query: obfuscation share, missing-RemoteUrl counts, completeness ratios, table population, // and the connections/detections exposed to proxy obfuscation. A true 0% obscured is a valid // finding (no obfuscation), not an error. // // NOTE // The obscured classification is only as complete as the proxy address set you supply (R5). // Advanced-hunting retention is limited — verify current retention. White-label. //
==========

How to run: // Set the two parameters at the top of each query (device sample + proxy address set), then run // the query you want. Nothing here changes the proxy, the agent, or the tenant. // // EXPECTED RESULTS // Per-query: obfuscation share, missing-RemoteUrl counts, completeness ratios, table population, // and the connections/detections exposed to proxy obfuscation. A true 0% obscured is a valid // finding (no obfuscation), not an error. // // NOTE // The obscured classification is only as complete as the proxy address set you supply (R5). // Advanced-hunting retention is limited — verify current retention. White-label. //
==========

MDEPROXY-Report.ps1

Purpose: Assembles 'MDEPROXY Assessment Report - Tenant - Validate' — the measurement summary (obfuscation %, completeness, table population), the feature-impact matrix scaffold (with the 'Confirm with Microsoft' column left OPEN), the XFF clarification, and the open questions for the Microsoft advisory case. It only READS the JSON that MDEPROXY-Validate.ps1 and MDEPROXY-FeatureImpact.ps1 produced — it queries nothing and changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDEPROXY-Report.ps1 `-BaselinePath .\MDEPROXY-telemetry-baseline.json` -FeaturePath .\MDEPROXY-feature-impact.json Writes a Markdown report (and a JSON sidecar). No tenant connection required. EXPECTED RESULTS MDEPROXY-assessment-report.md — a decision-ready evidence package that keeps every Microsoft-authoritative item as an OPEN question for the advisory case (never invented). .NOTES White-label. Read-only (reads local JSON only). Keeps the 'Confirm with Microsoft' framing intact. > [CmdletBinding()] param([string]$BaselinePath = ".\MDEPROXY-telemetry-baseline.json", [string]$FeaturePath = ".\MDEPROXY-feature-impact.json",

[string]$OutputPath = ".\MDEPROXY-assessment-report.md")

How to run: powershell -ExecutionPolicy Bypass -File .\MDEPROXY-Report.ps1 `-BaselinePath .\MDEPROXY-telemetry-baseline.json` -FeaturePath .\MDEPROXY-feature-impact.json Writes a Markdown report (and a JSON sidecar). No tenant connection required. EXPECTED RESULTS MDEPROXY-assessment-report.md — a decision-ready evidence package that keeps every Microsoft-authoritative item as an OPEN question for the advisory case (never invented). .NOTES White-label. Read-only (reads local JSON only). Keeps the 'Confirm with Microsoft' framing intact. > [CmdletBinding()] param([string]$BaselinePath = ".\MDEPROXY-telemetry-baseline.json", [string]$FeaturePath = ".\MDEPROXY-feature-impact.json", [string]$OutputPath = ".\MDEPROXY-assessment-report.md")

MDEPROXY-Validate.ps1

Purpose: Measures, over a device sample and window, HOW MUCH of the endpoint's destination telemetry the SWG proxy is obscuring: the share of outbound DeviceNetworkEvents whose RemoteIP falls in the loopback/proxy address set (obscured) vs a true remote, the count of connections with missing/empty RemoteUrl, and DeviceNetworkEvents completeness (network-to-process event ratio). Writes a telemetry baseline. Read-only advanced hunting only — changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDEPROXY-Validate.ps1 `-Sample @('AERO-WKS-01','AERO-WKS-02')` -ProxyAddressSet @('127.0.0.0/8','::1','10.10.10.5') -Days 7 -Sample accepts device names (array) OR a device-group/tag name resolved in DeviceInfo. -Check Obfuscation | Completeness | All (default All). Government cloud: -Environment. EXPECTED RESULTS MDEPROXY-telemetry-baseline.json — obfuscation %, missing-RemoteUrl count, per-device distribution, and completeness metrics. The script ECHOES the proxy address set it used so the classification is reviewable. A true 0% is a valid finding (no obfuscation), not an error. .NOTES White-label. Read-only. The finding is only as complete as the -ProxyAddressSet you supply — document every address (R5). Advanced-hunting retention is limited — verify current retention. > [CmdletBinding()] param([Parameter(Mandatory)] [string[]]$Sample, [string[]]$ProxyAddressSet = @('127.0.0.0/8','::1','169.254.0.0/16'), [int]$Days = 7, [ValidateSet('Obfuscation','Completeness','All')] [string]$Check = 'All', [string]$OutputPath = ".\MDEPROXY-telemetry-baseline.json", [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\MDEPROXY-Validate.ps1 `-Sample @('AERO-WKS-01','AERO-WKS-02')` -ProxyAddressSet @('127.0.0.0/8','::1','10.10.10.5') -Days 7 -Sample accepts device names (array) OR a device-group/tag name resolved in DeviceInfo. -Check Obfuscation | Completeness | All (default All). Government cloud: -Environment. EXPECTED RESULTS MDEPROXY-telemetry-baseline.json — obfuscation %, missing-RemoteUrl count, per-device distribution, and completeness metrics. The script ECHOES the proxy address set it used so the classification is reviewable. A true 0% is a valid finding (no obfuscation), not an error. .NOTES White-label. Read-only. The finding is only as complete as the -ProxyAddressSet you supply — document every address (R5). Advanced-hunting retention is limited — verify current retention. > [CmdletBinding()] param([Parameter(Mandatory)] [string[]]$Sample, [string[]]$ProxyAddressSet = @('127.0.0.0/8','::1','169.254.0.0/16'), [int]$Days = 7, [ValidateSet('Obfuscation','Completeness','All')] [string]$Check = 'All', [string]$OutputPath = ".\MDEPROXY-telemetry-baseline.json", [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')